

Loan Default Prediction using Machine Learning

Darshan University — Computer Engineering Department — ML Project

Dataset: Loan Default Prediction (Kaggle) —

<https://www.kaggle.com/datasets/nikhil1e9/loan-default>

Objective: Develop a machine learning classification model that predicts whether a borrower will default based on their financial, demographic and loan-related attributes.

Target: Default

Default = 0 → No Default

Default = 1 → Default

In []:

Problem Statement

Banks approve thousands of loans every year. Some customers fail to repay their loans, resulting in financial losses for the institution. The objective of this project is to develop a Machine Learning model that predicts whether a customer is **likely to default on a loan**, using their financial, employment, and demographic details. Early identification of high-risk applicants helps banks make better lending decisions and reduce non-performing assets (NPAs).

Dataset Summary

Item	Description
Dataset Name	Loan Default Prediction
Domain	Banking
Source	Kaggle
Target Variable	Default
Problem Type	Binary Classification (0 = No Default, 1 = Default)

Features used:

- Loan Amount
- Annual Income
- Age

- Employment Type
- Credit Score
- Loan Term
- Interest Rate
- Number of Dependents
- Existing Loans
- Property Area

Initial Observations

- The dataset contains a mix of **numerical** (Loan Amount, Income, Age, Credit Score...) and **categorical** (Employment Type, Property Area...) features.
- Missing values may be present and need to be handled.
- Some numerical features (e.g., Income, Loan Amount) are likely to contain outliers.
- The target variable `Default` is imbalanced in most real-world lending datasets (fewer defaulters than non-defaulters) — this will be checked and considered during evaluation.
- Data preprocessing (cleaning, encoding, scaling) is required before model training.

In []:

```
In [20]: # 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [21]: df = pd.read_csv('Loan_default.csv')
```

```
In [22]: df
```

Out[22]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
0	I38PQUQS96	56	85994	50587	520	80	
1	HPSK72WA7R	69	50432	124440	458	15	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	
3	V2KKSFM3UN	32	31713	44799	743	0	
4	EY08JDHTZP	60	20437	9139	633	8	
...	...	...	...	...	...	...	...
255342	8C6S86ESGC	19	37979	210682	541	109	
255343	98R4KDHNND	32	51953	189899	511	14	
255344	XQK1UUUNGP	56	84820	208294	597	70	
255345	JAO28CPL4H	42	85109	60575	809	40	
255346	ZTH91CGL0B	62	22418	18481	636	113	

255347 rows × 18 columns



In []:

In [23]:

df.head() # First 5 rows

Out[23]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCredit
0	I38PQUQS96	56	85994	50587	520	80	
1	HPSK72WA7R	69	50432	124440	458	15	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	
3	V2KKSFM3UN	32	31713	44799	743	0	
4	EY08JDHTZP	60	20437	9139	633	8	



In [6]:

df.tail() # Last 5 rows

Out[6]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nur
255342	8C6S86ESGC	19	37979	210682	541		109
255343	98R4KDHNNND	32	51953	189899	511		14
255344	XQK1UUUNGP	56	84820	208294	597		70
255345	JAO28CPL4H	42	85109	60575	809		40
255346	ZTH91CGL0B	62	22418	18481	636		113

In [6]: `df.sample()` # Random 5 Rows

Out[6]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumC
191072	CL7BFZZ7JZ	66	137083	9300	505		25

In [8]: `df.sample(5)`

Out[8]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	Nu
49266	QXUM1TR1WV	31	130905	190614	667		48
235943	A3XAXJ8U2W	35	122374	84364	306		23
6842	QY5CBEK9OK	45	40145	215701	414		16
74680	U5E6VVESTI	26	124737	84600	767		25
47393	X0HOFX5N9E	48	41310	36236	383		54

In [7]: `df.shape` # Dataset Shape

Out[7]: (255347, 18)

In [7]: `print("Rows :", df.shape[0])`
`print("Columns :", df.shape[1])`

Rows : 255347
Columns : 18

In [8]: `df.index` # Dataset Index

Out[8]: RangeIndex(start=0, stop=255347, step=1)

In [9]: `df.info()` # Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 255347 entries, 0 to 255346
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   LoanID                 255347 non-null object
1   Age                    255347 non-null int64
2   Income                 255347 non-null int64
3   LoanAmount             255347 non-null int64
4   CreditScore            255347 non-null int64
5   MonthsEmployed         255347 non-null int64
6   NumCreditLines         255347 non-null int64
7   InterestRate           255347 non-null float64
8   LoanTerm               255347 non-null int64
9   DTIRatio               255347 non-null float64
10  Education              255347 non-null object
11  EmploymentType         255347 non-null object
12  MaritalStatus          255347 non-null object
13  HasMortgage            255347 non-null object
14  HasDependents          255347 non-null object
15  LoanPurpose            255347 non-null object
16  HasCoSigner            255347 non-null object
17  Default                255347 non-null int64
dtypes: float64(2), int64(8), object(8)
memory usage: 35.1+ MB
```

```
In [11]: df.dtypes # Data Types
```

```
Out[11]: LoanID                object
Age                  int64
Income               int64
LoanAmount           int64
CreditScore          int64
MonthsEmployed       int64
NumCreditLines       int64
InterestRate         float64
LoanTerm             int64
DTIRatio             float64
Education            object
EmploymentType       object
MaritalStatus        object
HasMortgage          object
HasDependents        object
LoanPurpose          object
HasCoSigner          object
Default              int64
dtype: object
```

```
In [13]: df.columns # Column Names
```

```
Out[13]: Index(['LoanID', 'Age', 'Income', 'LoanAmount', 'CreditScore',
                'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm',
                'DTIRatio', 'Education', 'EmploymentType', 'MaritalStatus',
                'HasMortgage', 'HasDependents', 'LoanPurpose', 'HasCoSigner',
                'Default'],
                dtype='object')
```

```
In [14]: df.nunique() # Number of Unique Values
```

```
Out[14]: LoanID      255347
Age          52
Income       114620
LoanAmount   158729
CreditScore  550
MonthsEmployed 120
NumCreditLines 4
InterestRate 2301
LoanTerm     5
DTIRatio     81
Education    4
EmploymentType 4
MaritalStatus 3
HasMortgage  2
HasDependents 2
LoanPurpose  5
HasCoSigner  2
Default      2
dtype: int64
```

```
In [15]: df.describe() # Categorical Statistical Summary
```

```
Out[15]:
```

	Age	Income	LoanAmount	CreditScore	MonthsEmployed
count	255347.000000	255347.000000	255347.000000	255347.000000	255347.000000
mean	43.498306	82499.304597	127578.865512	574.264346	59.541976
std	14.990258	38963.013729	70840.706142	158.903867	34.643376
min	18.000000	15000.000000	5000.000000	300.000000	0.000000
25%	31.000000	48825.500000	66156.000000	437.000000	30.000000
50%	43.000000	82466.000000	127556.000000	574.000000	60.000000
75%	56.000000	116219.000000	188985.000000	712.000000	90.000000
max	69.000000	149999.000000	249999.000000	849.000000	119.000000

```
In [20]: df.describe(include='object') # Categorical Statistical Summary
```

```
Out[20]:
```

	LoanID	Education	EmploymentType	MaritalStatus	HasMortgage	HasDe
count	255347	255347	255347	255347	255347	
unique	255347	4	4	3	2	
top	ZTH91CGL0B	Bachelor's	Part-time	Married	Yes	
freq	1	64366	64161	85302	127677	

```
In [26]: df['Default'].value_counts() # Target Distribution
```

```
Out[26]: Default
0      225694
1       29653
Name: count, dtype: int64
```

```
In [6]: # Target Percentage
df['Default'].value_counts(normalize=True) * 100
```

```
Out[6]: Default
0      88.387175
1      11.612825
Name: proportion, dtype: float64
```

```
In [ ]: # 88.39% → No Default
# 11.61% → Default
```

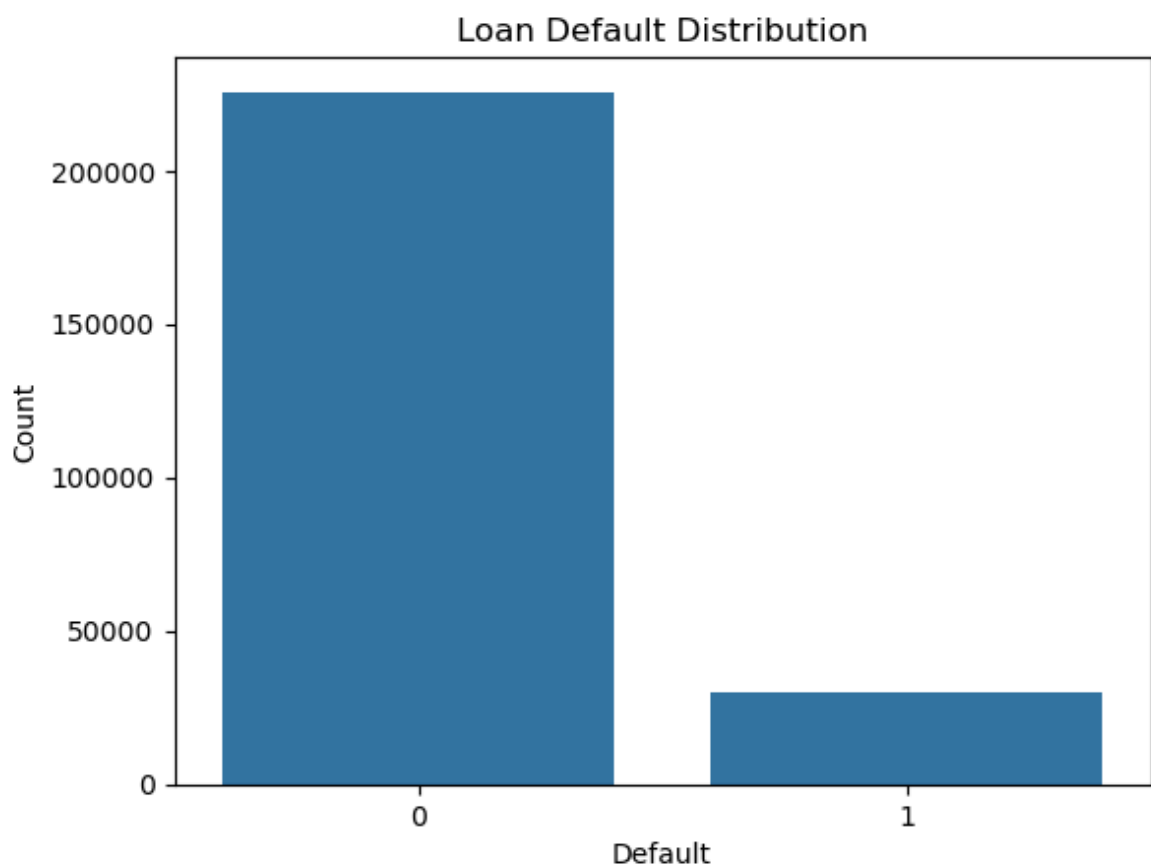
```
In [ ]:
```

```
In [24]: # Target Visualization
sns.countplot(x='Default', data=df)

plt.title('Loan Default Distribution')
plt.xlabel('Default')
plt.ylabel('Count')

plt.show()

# No Default vs Default
```



```
In [41]: # Missing Values
df.isnull().sum()
```

```
Out[41]: LoanID      0
         Age        0
         Income     0
         LoanAmount 0
         CreditScore 0
         MonthsEmployed 0
         NumCreditLines 0
         InterestRate 0
         LoanTerm    0
         DTIRatio    0
         Education   0
         EmploymentType 0
         MaritalStatus 0
         HasMortgage 0
         HasDependents 0
         LoanPurpose 0
         HasCoSigner 0
         Default     0
         dtype: int64
```

```
In [44]: # Missing Value Percentage
         df.isnull().mean() * 100
```

```
Out[44]: LoanID      0.0
         Age        0.0
         Income     0.0
         LoanAmount 0.0
         CreditScore 0.0
         MonthsEmployed 0.0
         NumCreditLines 0.0
         InterestRate 0.0
         LoanTerm    0.0
         DTIRatio    0.0
         Education   0.0
         EmploymentType 0.0
         MaritalStatus 0.0
         HasMortgage 0.0
         HasDependents 0.0
         LoanPurpose 0.0
         HasCoSigner 0.0
         Default     0.0
         dtype: float64
```

```
In [45]: # Duplicate Rows
         df.duplicated().sum()
```

```
Out[45]: np.int64(0)
```

```
In [52]: # Numerical Median
         df.median(numeric_only=True)
```

```
Out[52]: Age                43.00
Income              82466.00
LoanAmount         127556.00
CreditScore        574.00
MonthsEmployed     60.00
NumCreditLines      2.00
InterestRate       13.46
LoanTerm           36.00
DTIRatio           0.50
Default            0.00
dtype: float64
```

```
In [53]: # Numerical Mode
df.mode(numeric_only=True).iloc[0]
```

```
Out[53]: Age                55.00
Income              69492.00
LoanAmount         95419.00
CreditScore        630.00
MonthsEmployed     56.00
NumCreditLines      2.00
InterestRate       14.92
LoanTerm           48.00
DTIRatio           0.67
Default            0.00
Name: 0, dtype: float64
```

```
In [54]: # Minimum Values
df.min(numeric_only=True)
```

```
Out[54]: Age                18.0
Income              15000.0
LoanAmount         5000.0
CreditScore        300.0
MonthsEmployed      0.0
NumCreditLines      1.0
InterestRate        2.0
LoanTerm           12.0
DTIRatio           0.1
Default            0.0
dtype: float64
```

```
In [55]: # Maximum Values
df.max(numeric_only=True)
```

```
Out[55]: Age                69.0
Income             149999.0
LoanAmount        249999.0
CreditScore       849.0
MonthsEmployed    119.0
NumCreditLines     4.0
InterestRate      25.0
LoanTerm          60.0
DTIRatio          0.9
Default           1.0
dtype: float64
```

```
In [56]: # Standard Deviation
df.std(numeric_only=True)
```

```
Out[56]: Age                14.990258
Income            38963.013729
LoanAmount        70840.706142
CreditScore       158.903867
MonthsEmployed    34.643376
NumCreditLines    1.117018
InterestRate      6.636443
LoanTerm          16.969330
DTIRatio          0.230917
Default           0.320379
dtype: float64
```

```
In [57]: # Memory Usage
df.memory_usage(deep=True)
```

```
Out[57]: Index                132
LoanID              15065473
Age                 2042776
Income              2042776
LoanAmount          2042776
CreditScore         2042776
MonthsEmployed      2042776
NumCreditLines      2042776
InterestRate        2042776
LoanTerm            2042776
DTIRatio            2042776
Education           14557535
EmploymentType      15128774
MaritalStatus       14299453
HasMortgage         13150374
HasDependents       13150439
LoanPurpose         14044522
HasCoSigner         13150398
Default             2042776
dtype: int64
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Week 2 — Data Cleaning, Preprocessing & EDA

```
In [25]: # Create Clean Copy
df_clean = df.copy()
```

```
In [26]: # Check Missing Values
print("Total Missing Values:",
      df_clean.isnull().sum().sum())

df_clean.isnull().sum()
```

Total Missing Values: 0

```
Out[26]: LoanID      0
         Age         0
         Income      0
         LoanAmount  0
         CreditScore 0
         MonthsEmployed 0
         NumCreditLines 0
         InterestRate 0
         LoanTerm     0
         DTIRatio     0
         Education    0
         EmploymentType 0
         MaritalStatus 0
         HasMortgage  0
         HasDependents 0
         LoanPurpose  0
         HasCoSigner  0
         Default      0
         dtype: int64
```

```
In [61]: # Check Duplicate Rows
         print("Duplicate Rows:",
               df_clean.duplicated().sum())
```

Duplicate Rows: 0

```
In [62]: # Remove Duplicate Rows
         df_clean.drop_duplicates(inplace=True)
```

```
In [63]: # Check Data Types
         df_clean.dtypes
```

```
Out[63]: LoanID      object
         Age         int64
         Income      int64
         LoanAmount  int64
         CreditScore int64
         MonthsEmployed int64
         NumCreditLines int64
         InterestRate float64
         LoanTerm     int64
         DTIRatio     float64
         Education    object
         EmploymentType object
         MaritalStatus object
         HasMortgage  object
         HasDependents object
         LoanPurpose  object
         HasCoSigner  object
         Default      int64
         dtype: object
```

```
In [27]: # Identify Numerical Columns
         numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()

         numeric_cols.remove('Default')

         print(numeric_cols)
```

```
['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed', 'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio']
```

```
In [29]: # Remove LoanID
df_clean.drop('LoanID', axis=1, inplace=True)
```

```
In [30]: # Check Shape After Removing LoanID
print("Rows      :", df_clean.shape[0])
print("Columns   :", df_clean.shape[1])
```

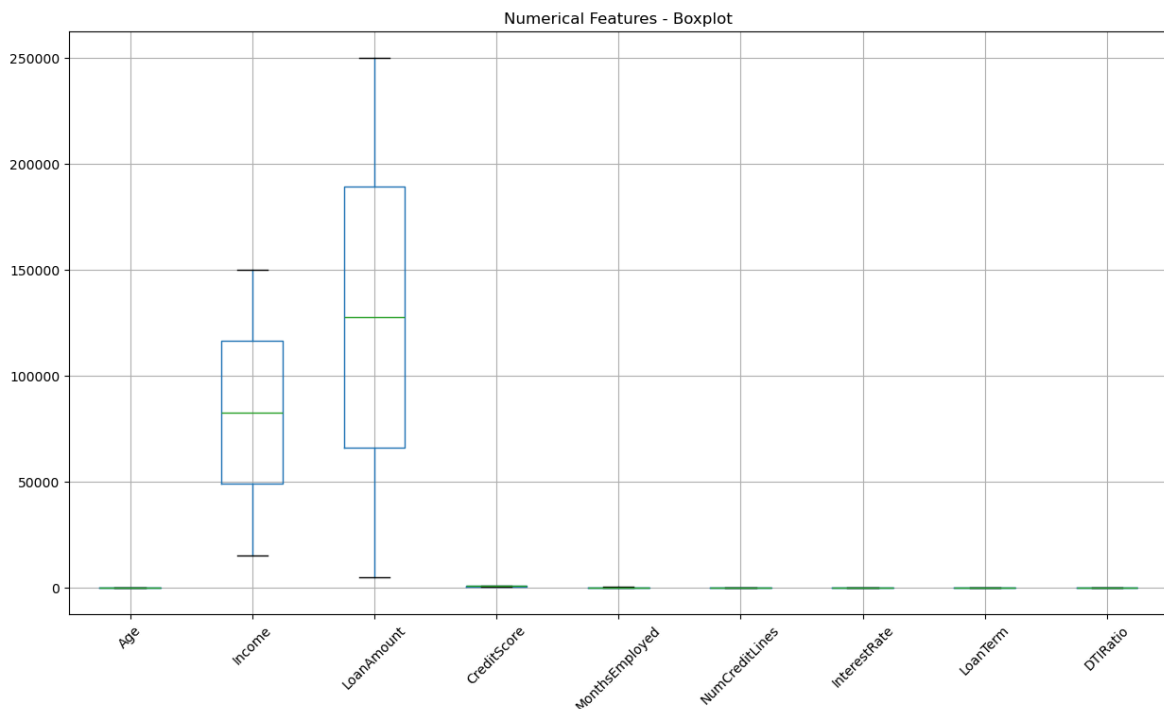
```
Rows      : 255347
Columns   : 17
```

```
In [31]: # Numerical Boxplots
plt.figure(figsize=(15, 8))

df_clean[numeric_cols].boxplot()

plt.xticks(rotation=45)
plt.title('Numerical Features - Boxplot')

plt.show()
```



IQR Method

```
In [32]: # IQR Outlier Detection
for col in numeric_cols:

    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
```

```
outliers = df_clean[
    (df_clean[col] < lower) |
    (df_clean[col] > upper)
]

print(col, ":", len(outliers))
```

Age : 0
Income : 0
LoanAmount : 0
CreditScore : 0
MonthsEmployed : 0
NumCreditLines : 0
InterestRate : 0
LoanTerm : 0
DTIRatio : 0

Outliers were checked using the IQR method. No IQR-based outliers were detected in the numerical features. Therefore, no rows were removed due to outliers.

In []:

Categorical Variables

```
In [33]: categorical_cols = df_clean.select_dtypes(include='object').columns

print(categorical_cols)
```

```
Index(['Education', 'EmploymentType', 'MaritalStatus', 'HasMortgage',
      'HasDependents', 'LoanPurpose', 'HasCoSigner'],
      dtype='object')
```

```
In [34]: # Categorical Value Counts
for col in categorical_cols:
    print("-----\n", col)
    print(df_clean[col].value_counts())
```

```
-----
Education
Education
Bachelor's      64366
High School    63903
Master's       63541
PhD            63537
Name: count, dtype: int64
-----
```

```
-----
EmploymentType
EmploymentType
Part-time      64161
Unemployed     63824
Self-employed  63706
Full-time      63656
Name: count, dtype: int64
-----
```

```
-----
MaritalStatus
MaritalStatus
Married        85302
Divorced       85033
Single         85012
Name: count, dtype: int64
-----
```

```
-----
HasMortgage
HasMortgage
Yes            127677
No             127670
Name: count, dtype: int64
-----
```

```
-----
HasDependents
HasDependents
Yes            127742
No             127605
Name: count, dtype: int64
-----
```

```
-----
LoanPurpose
LoanPurpose
Business       51298
Home           51286
Education      51005
Other          50914
Auto           50844
Name: count, dtype: int64
-----
```

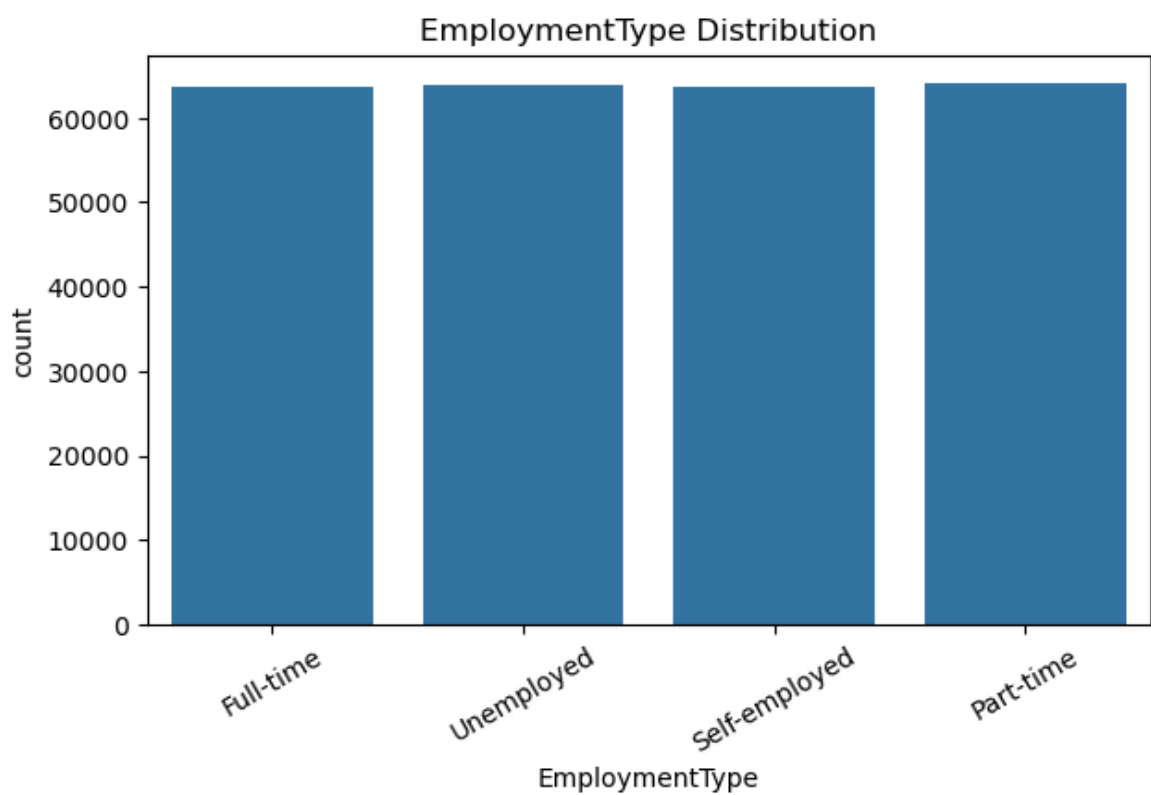
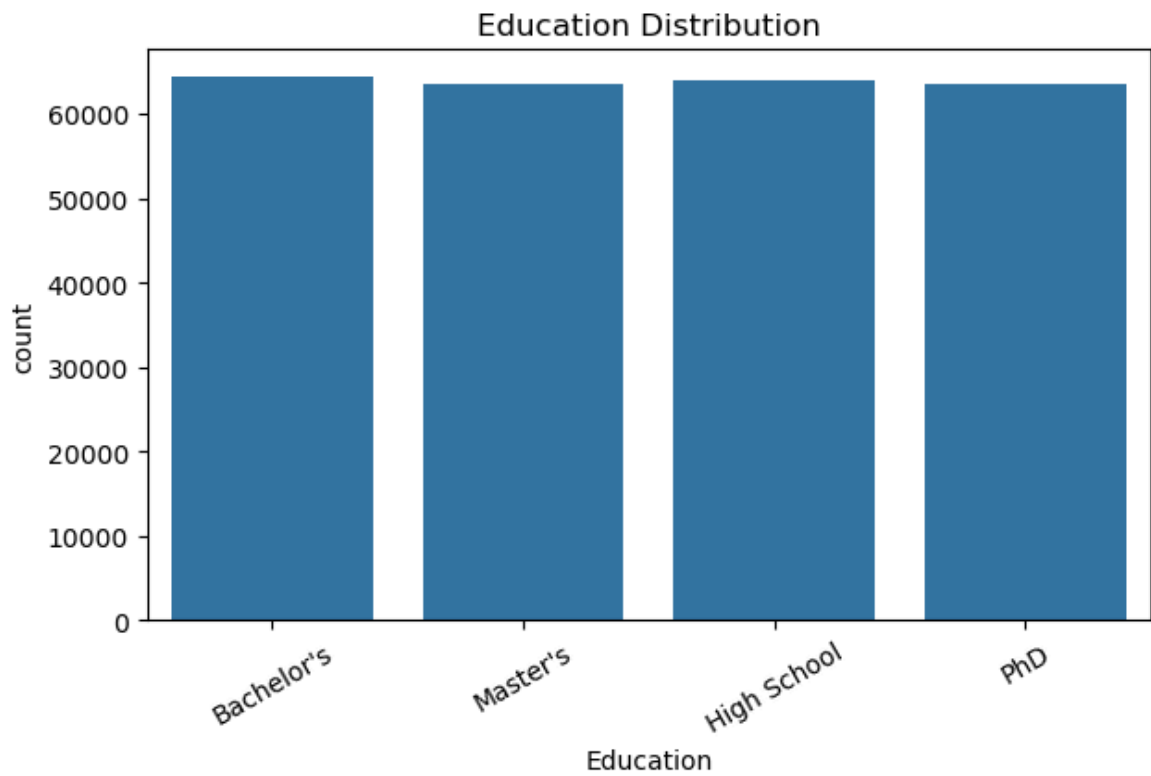
```
-----
HasCoSigner
HasCoSigner
Yes            127701
No             127646
Name: count, dtype: int64
-----
```

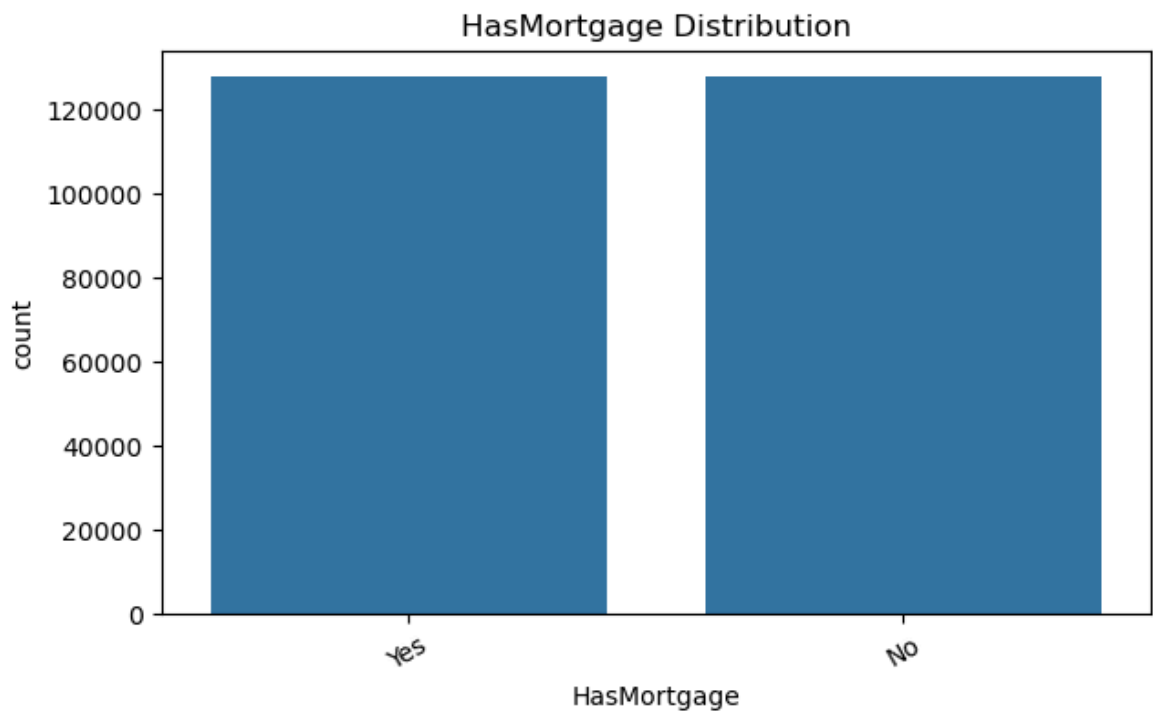
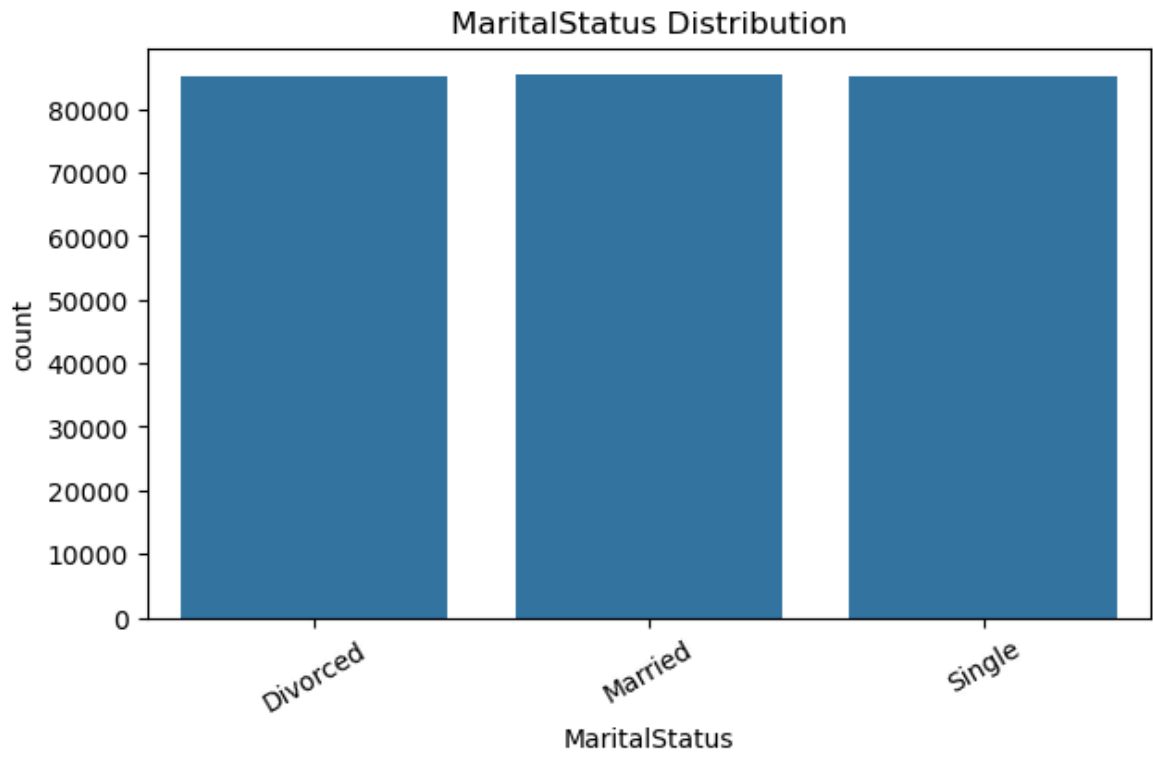
```
In [35]: # Categorical Countplots
for col in categorical_cols:

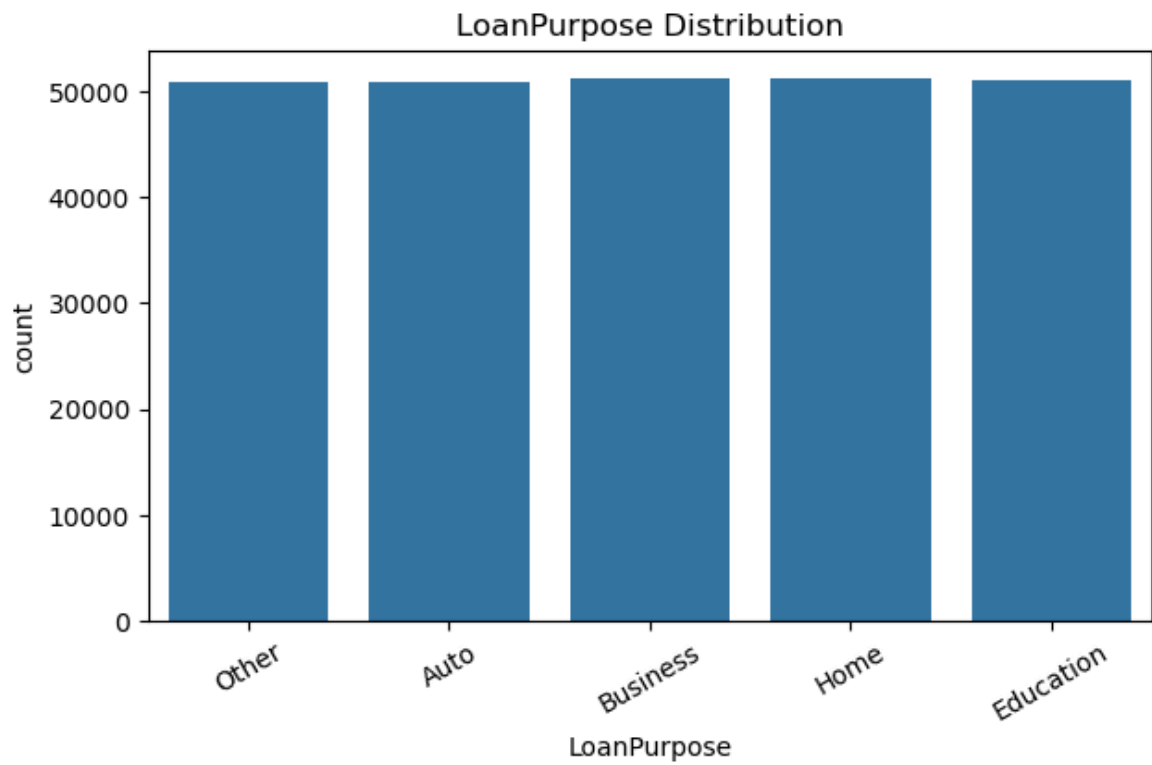
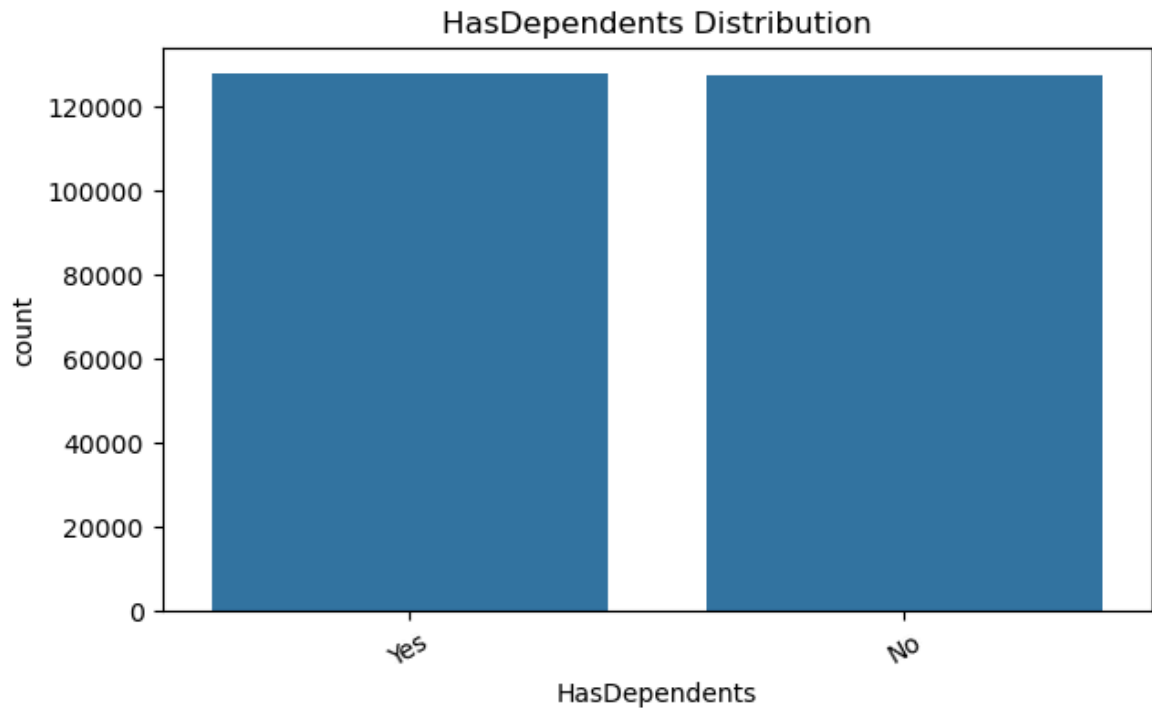
    plt.figure(figsize=(7, 4))

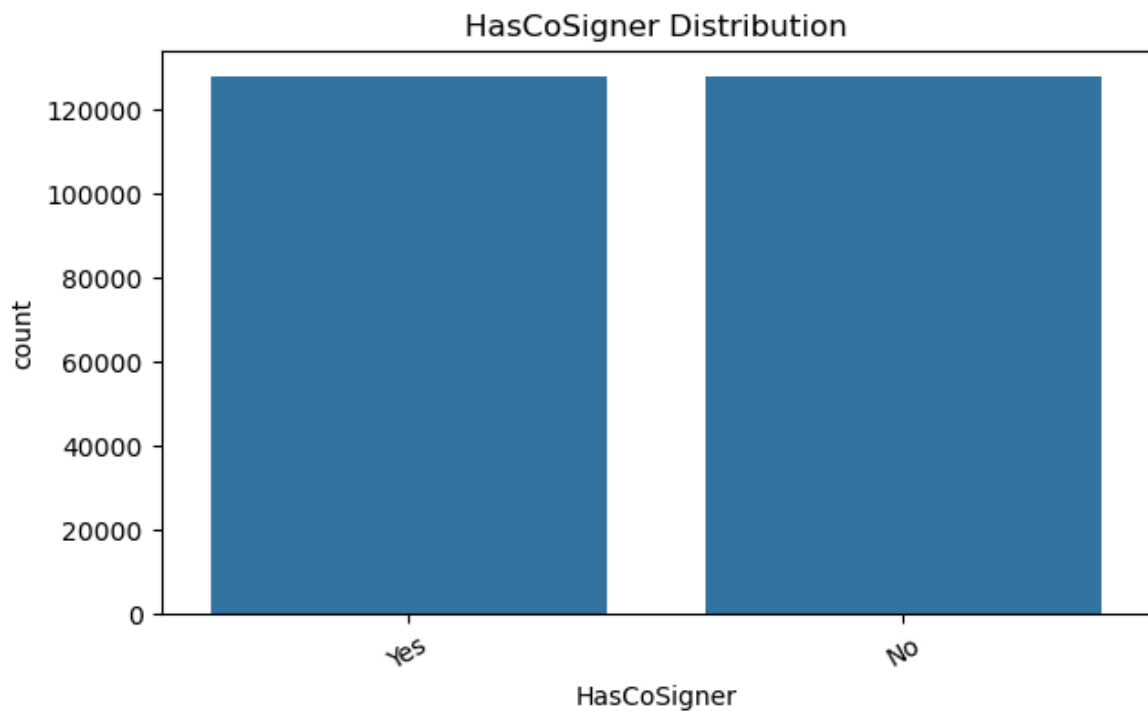
    sns.countplot(
        x=col,
        data=df_clean
    )
```

```
plt.title(f'{col} Distribution')  
plt.xticks(rotation=30)  
  
plt.show()
```









```
In [78]: # One-Hot Encoding
df_encoded = pd.get_dummies(
    df_clean,
    columns=categorical_cols,
    drop_first=True
)
```

```
In [79]: # Check Encoded Dataset
df_encoded.head()
```

```
Out[79]:
```

	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines	Interest
--	-----	--------	------------	-------------	----------------	----------------	----------

0	56	85994	50587	520	80	4	
1	69	50432	124440	458	15	1	
2	46	84208	129188	451	26	3	
3	32	31713	44799	743	0	3	
4	60	20437	9139	633	8	4	



```
In [80]: # Check Encoded Columns
df_encoded.columns
```

```
Out[80]: Index(['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEmployed',
               'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio', 'Default',
               'Education_High School', 'Education_Master's', 'Education_PhD',
               'EmploymentType_Part-time', 'EmploymentType_Self-employed',
               'EmploymentType_Unemployed', 'MaritalStatus_Married',
               'MaritalStatus_Single', 'HasMortgage_Yes', 'HasDependents_Yes',
               'LoanPurpose_Business', 'LoanPurpose_Education', 'LoanPurpose_Home',
               'LoanPurpose_Other', 'HasCoSigner_Yes'],
              dtype='object')
```

```

In [81]: # Numerical Columns for Scaling
scale_cols = df_encoded.select_dtypes(
    include=np.number
).columns.tolist()

scale_cols.remove('Default')

In [82]: # Standard Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df_encoded[scale_cols] = scaler.fit_transform(
    df_encoded[scale_cols]
)

In [83]: # Check Scaled Data
df_encoded[scale_cols].head()

```

```

Out[83]:

```

	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines
0	0.833990	0.089693	-1.086833	-0.341492	0.590533	1.341937
1	1.701221	-0.823021	-0.044309	-0.731666	-1.285731	-1.343791
2	0.166888	0.043854	0.022715	-0.775718	-0.968209	0.446694
3	-0.767053	-1.303452	-1.168538	1.061875	-1.718715	0.446694
4	1.100830	-1.592855	-1.671921	0.369631	-1.487790	1.341937

```

In [ ]:

```

```

In [ ]:

```

EDA

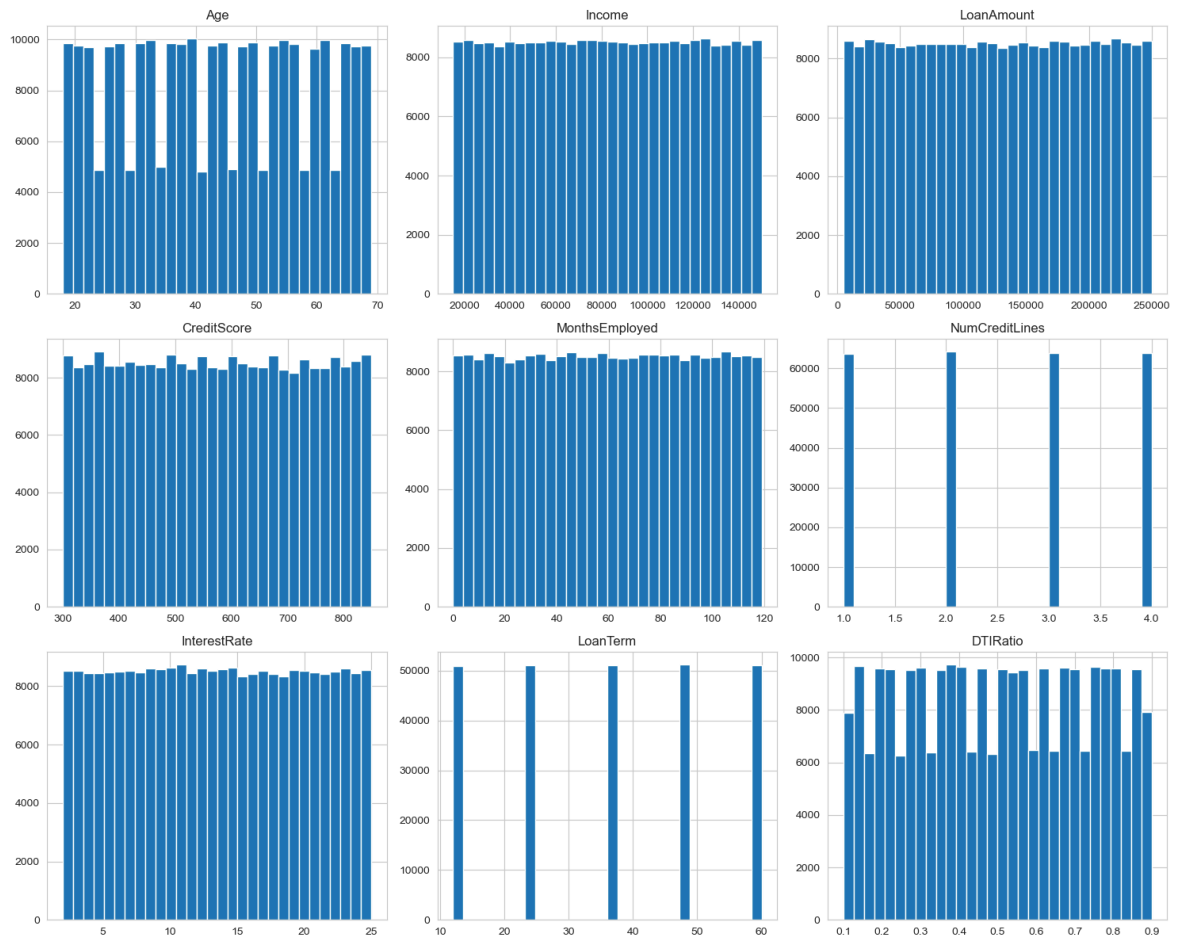
Numerical Distribution

```

In [85]: # Numerical Histograms
df_clean[numeric_cols].hist(
    figsize=(15, 12),
    bins=30
)

plt.tight_layout()
plt.show()

```



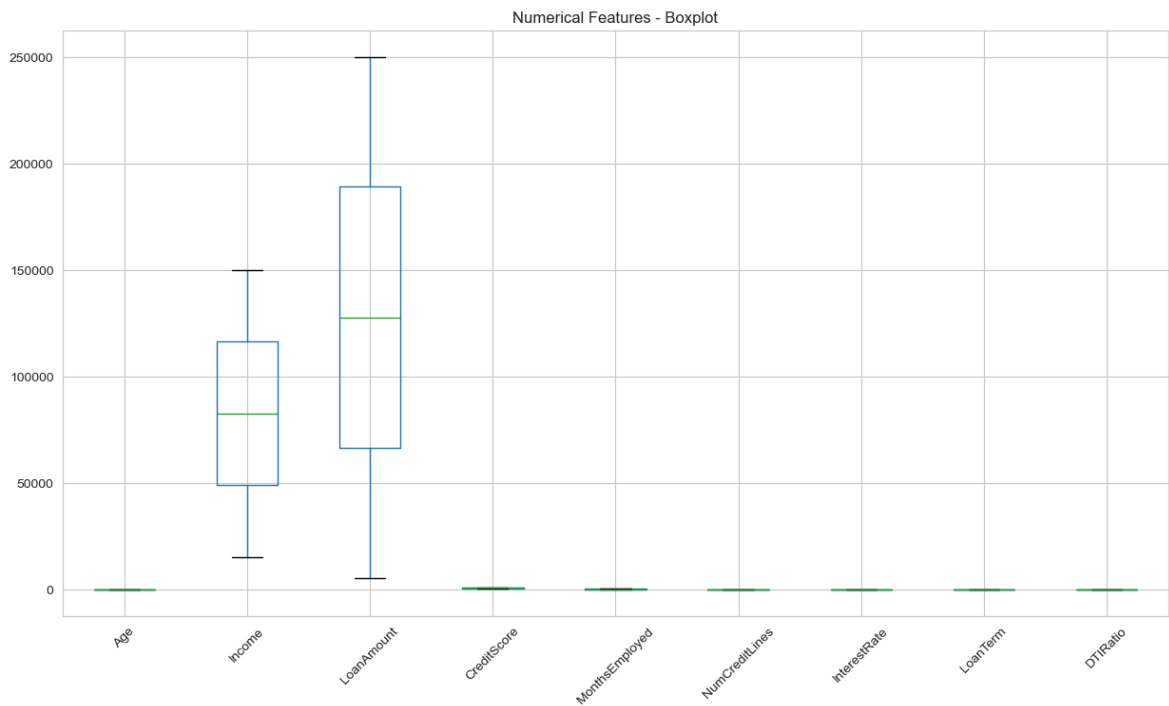
Boxplots

```
In [86]: # Numerical Boxplots
plt.figure(figsize=(15, 8))

df_clean[numeric_cols].boxplot()

plt.xticks(rotation=45)
plt.title('Numerical Features - Boxplot')

plt.show()
```



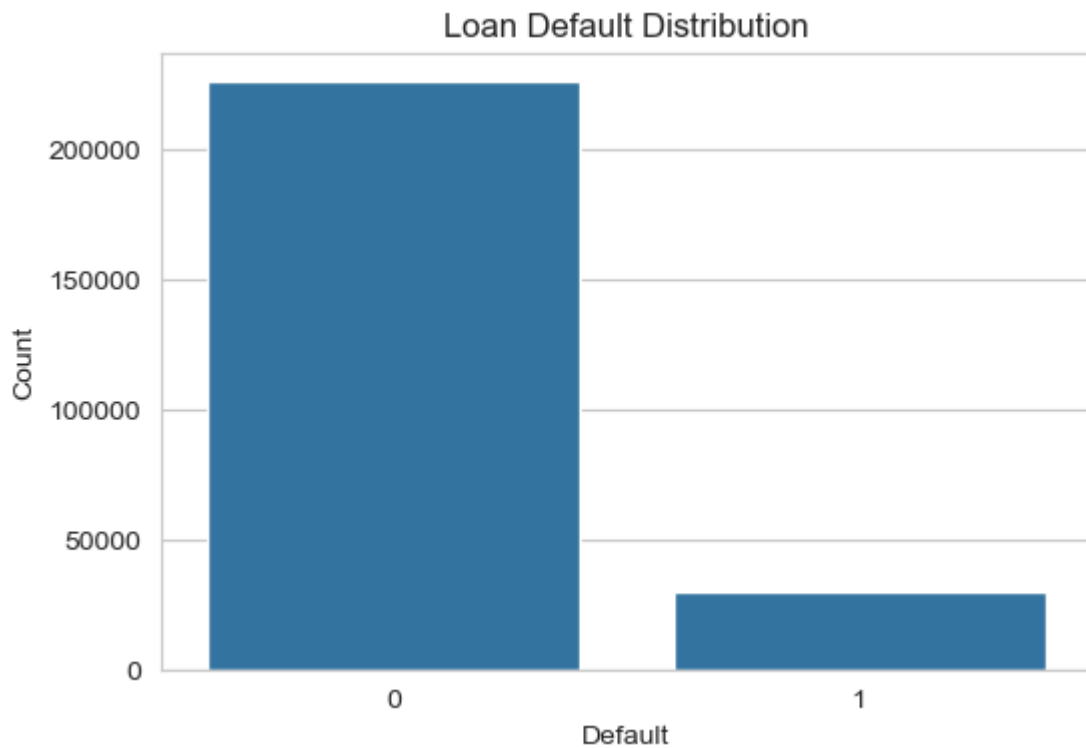
Default Distribution

```
In [87]: # Default Distribution
plt.figure(figsize=(6, 4))

sns.countplot(
    x='Default',
    data=df_clean
)

plt.title('Loan Default Distribution')
plt.xlabel('Default')
plt.ylabel('Count')

plt.show()
```



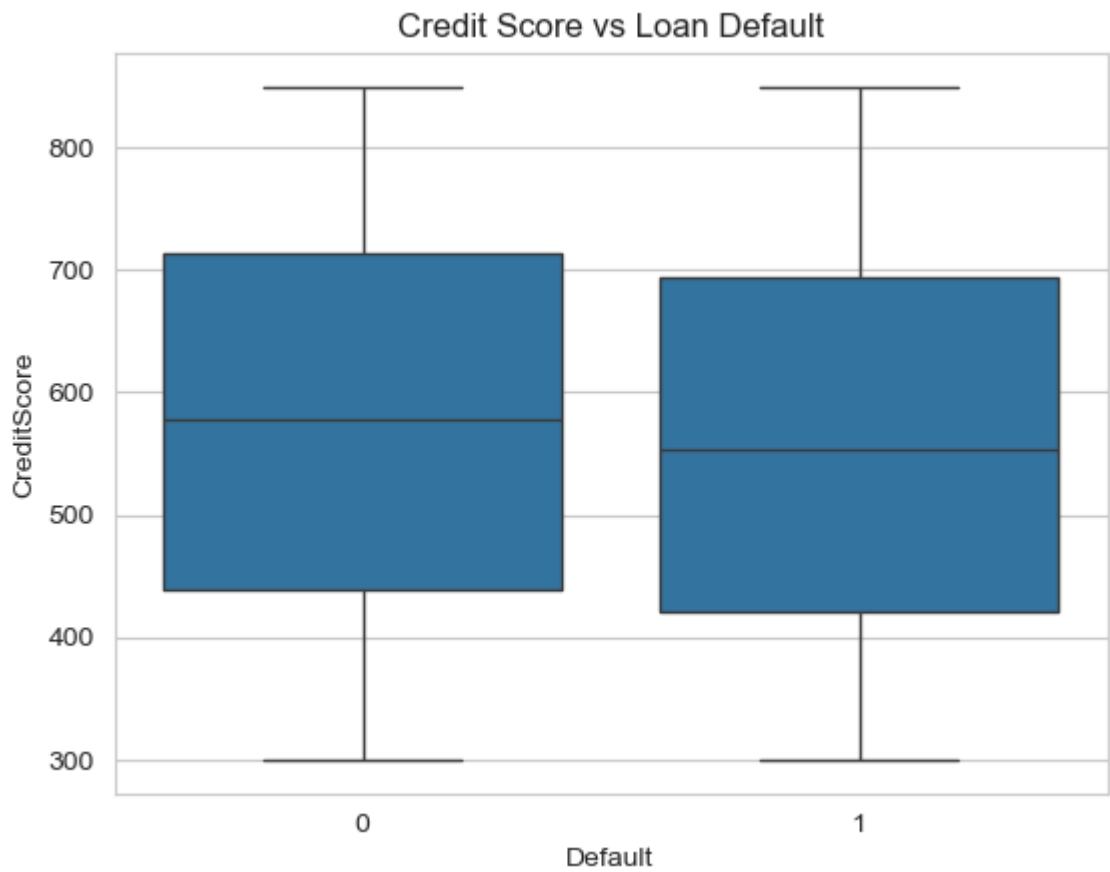
EDA

Credit Score vs Default

```
In [89]: # Credit Score vs Default
sns.boxplot(
    x='Default',
    y='CreditScore',
    data=df_clean
)

plt.title('Credit Score vs Loan Default')

plt.show()
```

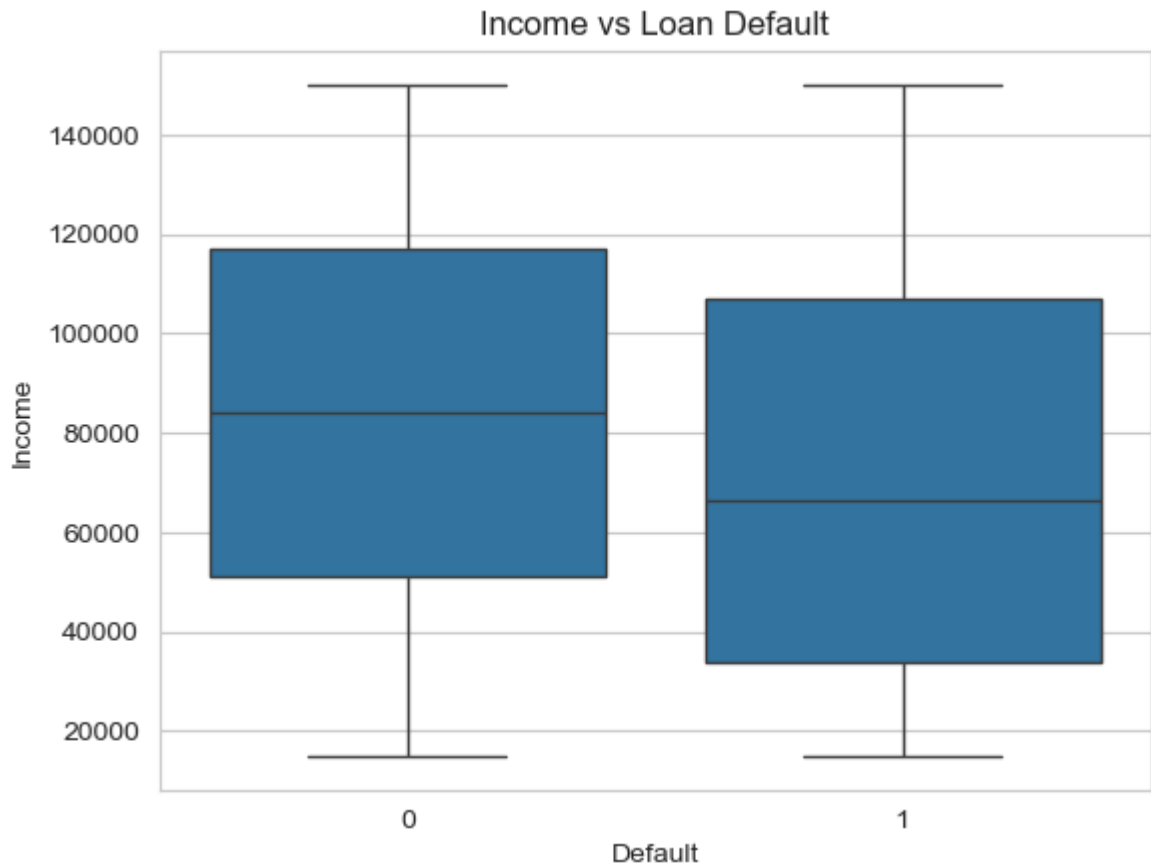


Income vs Default

```
In [91]: # Income vs Default
sns.boxplot(
    x='Default',
    y='Income',
    data=df_clean
)

plt.title('Income vs Loan Default')

plt.show()
```

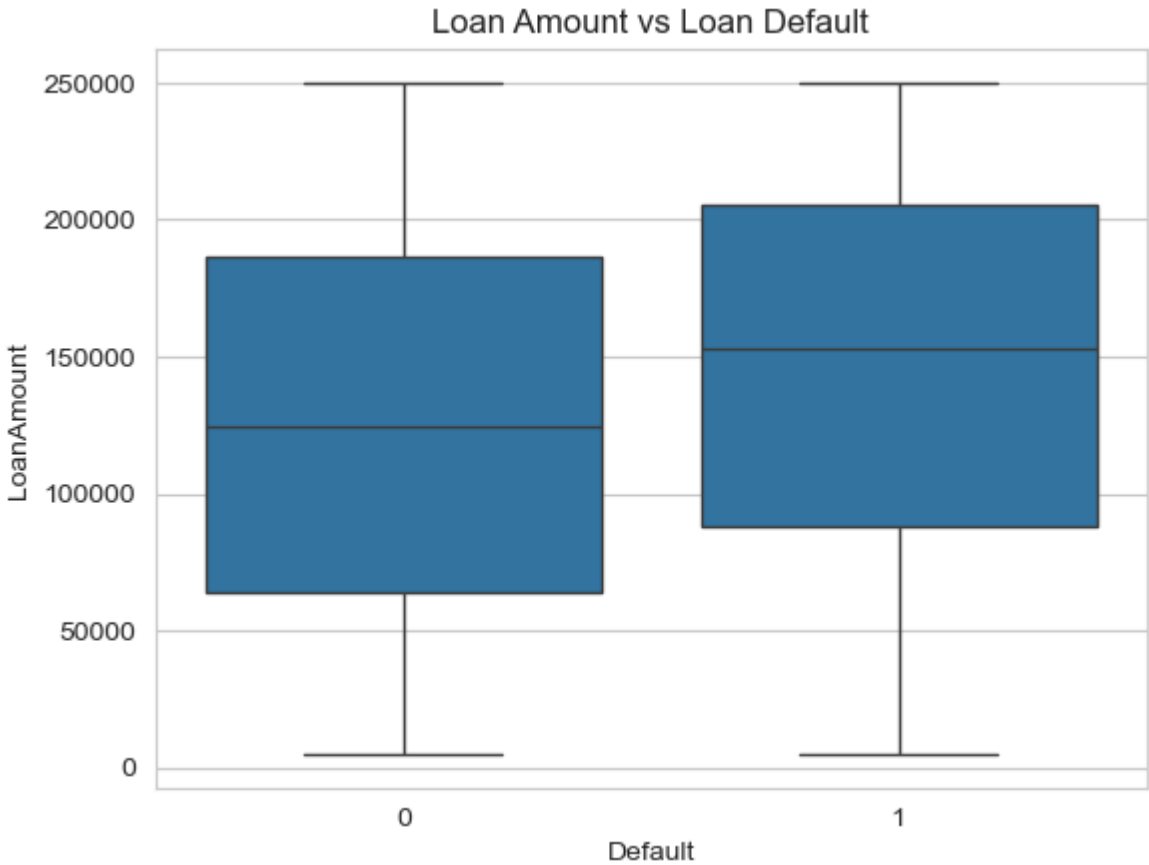


Loan Amount vs Default

```
In [92]: # Loan Amount vs Default
sns.boxplot(
    x='Default',
    y='LoanAmount',
    data=df_clean
)

plt.title('Loan Amount vs Loan Default')

plt.show()
```



In []:

In []: